

Lab Work 03

Topic	Inheritance, <code>super()</code> , Method Overriding, Polymorphism, Abstract Classes (abc)
Objective	Implement inheritance hierarchies, override methods, demonstrate polymorphism, and understand the use of abstract base classes.
AI Copilot Usage	Not Allowed

Tasks

1. Create a Base *Person* Class:

- Define a new base class *Person*.
- Give it protected attributes `_first_name` and `_last_name` initialized via the constructor.
- Include a `display_info()` method that prints the person's full name.

2. Implement Inheritance:

- Modify the *Student* class to inherit from *Person*.
- Use `super().__init__(first_name, last_name)` in the *Student* constructor to initialize the inherited attributes. Keep the `__student_id` specific to *Student*.
- Modify the *Teacher* class to inherit from *Person*. Use `super()` similarly. Keep `__employee_id` and `_subject_taught` specific to *Teacher*.
- Remove `_first_name`, `_last_name` definitions from *Student* and *Teacher* as they are now inherited.
- Adjust methods like `get_full_name` if necessary (it might be better to have it in the *Person* class now). Test creating *Student* and *Teacher* objects.

3. Method Overriding and Polymorphism:

- Override the `display_info()` method in both *Student* and *Teacher*.
- The *Student.display_info()* should call `super().display_info()` and then print the Student ID.
- The *Teacher.display_info()* should call `super().display_info()` and then print the Employee ID and Subject Taught.
- Create a list containing a mix of *Student* and *Teacher* objects.
- Iterate through the list and call the `display_info()` method on each object. Observe how the correct overridden method is called for each object type (polymorphism).

4. Abstract Base Class (ABC):

- a) Import **ABC** and **abstractmethod** from the **abc** module.
 - b) Modify the **Person** class to inherit from **ABC**.
 - c) Add an abstract method **get_role(self)** to the **Person** class using the **@abstractmethod** decorator.
 - d) Implement the **get_role()** method in both **Student** (returning "Student") and **Teacher** (returning "Teacher").
 - e) Try instantiating a **Person** object directly. Observe the **TypeError**.
5. **Creative Challenge (Optional, if time permits):**
- a) Create specialized student types based on the initial design brief: **UndergraduateStudent** and **PostgraduateStudent**, both inheriting from **Student**.
 - b) Add specific attributes (e.g., **_major** for Undergraduate, **_research_topic** for Postgraduate).
 - c) Override the **display_info** method again in these subclasses to include the specific attribute.
 - d) Add these new student types to your list of people and test the polymorphic **display_info** call.

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.